

Software Architecture Stage 1

E-Commerce Web Application

Zehra KOLAT 206001007

Göktuğ Şahin 2309111082

Kutay Şahinler 220911757

SWE312 - Software Architecture

Instructor: BAHMAN ARASTEH ABBASABAD



Istinye University

Faculty of Engineering and Natural Sciences

Software Engineering Department

May 2026

Software Architecture Stage 1

E-Commerce Web Application

Zehra KOLAT 206001007

Göktuğ Şahin 2309111082

Kutay Şahinler 220911757

01/05/2026

Contents

1	Contributions	4
2	System Selection : E-Commerce Web Application	4
2.1	Users:	5
2.1.1	Customer	5
2.1.2	Admin	5
2.2	Functionalities	5
3	Software Architecture Document (SAD) Version 1	6
3.1	Use Case View:	6
3.2	Logical View:	7
3.3	Process View:	8
4	The CODE Part	9
4.1	Partial UI Code	9
	index html - Login screen	9
	register html - registration screen	9
	Products html	10
	admin html	11
	admin.js	11
	script file-The logic behind form:	12
	Products.js	14
	css file- The design of both pages is shared:	16
4.2	Backend Code	19
4.2.1	controllers	19
	authController.js	19

	cartController.js	20
	productController.js	21
4.2.2	models	22
	cartModel	22
	productModel	23
	userModel	24
4.2.3	server.js	24
5	Web Application	25
5.1	User/Customer login	25
5.1.1	Admin login	27

1 Contributions

- Zehra Kolat:
 - Use Case Diagram
 - Process View
 - Logical View
 - Documentation
 - Postman:

The backend code was tested to see if it was working with Postman and it was confirmed to be working. For negative testing, when a user with an account tried to sign up again, they received a warning such as "400 Bad Request User already exists" or when entering an incorrect password, they received a warning such as "401 Unauthorized Invalid email or password". The backend health check was also successfully completed.

- Göktuğ Şahin:
 - Backend Code
- Kutay Şahinler:
 - Frontend UI

2 System Selection : E-Commerce Web Application

In this system, users can view products, add them to their cart, and place orders. Admins can manage the products.

2.1 Users:

2.1.1 Customer

- views products
- adds to cart
- places order
- views order history

2.1.2 Admin

- adds products
- deletes/updates products
- views orders

2.2 Functionalities

- Register / Login
- Product listing
- Product detail
- Add to cart
- Create order
- View order history
- Admin product management

3 Software Architecture Document (SAD) Version 1

3.1 Use Case View:

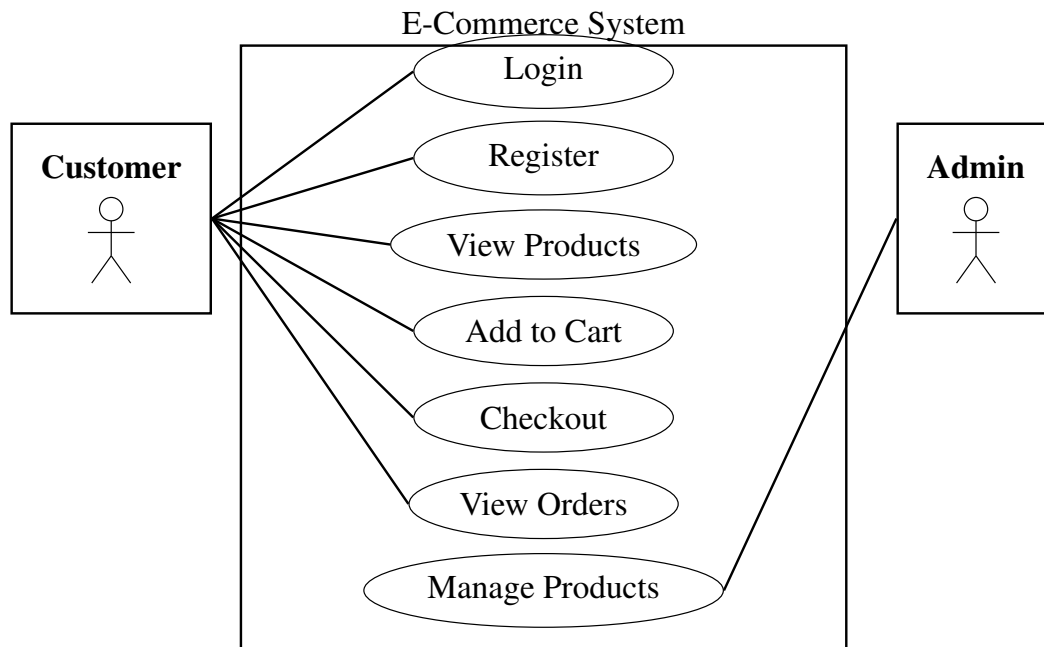


Figure 1: Use Case Diagram for E-Commerce System

The use case view shows the main functions of the system and the interactions between users and the system. Here, we describe the basic functions of an e-commerce system from the user's perspective. Customers can browse products, add products to their cart, make payments, and view their orders. Administrators can manage products and view orders.

3.2 Logical View:

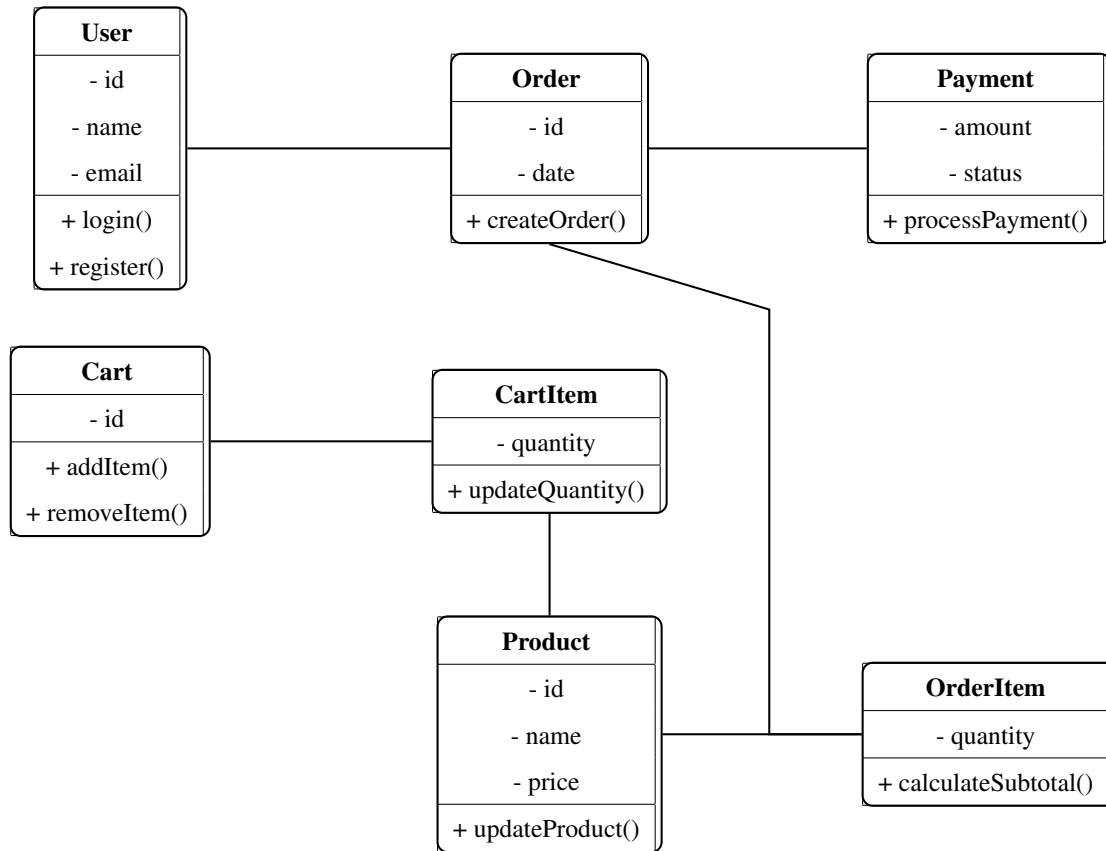


Figure 2: Logical View - Class Diagram

- User has many Orders.
- Order has many OrderItems.
- Cart has many CartItems.
- CartItem belongs to Product.
- Product appears in many OrderItems.

The logical view represents the basic structure of the system: it shows the main entities and relationships of the system. The core entities are User, Product, Cart, Order, and Payment. These entities represent the business logic of the e-commerce application. The relationships between these entities define how data is organized and how different parts of the system interact.

3.3 Process View:

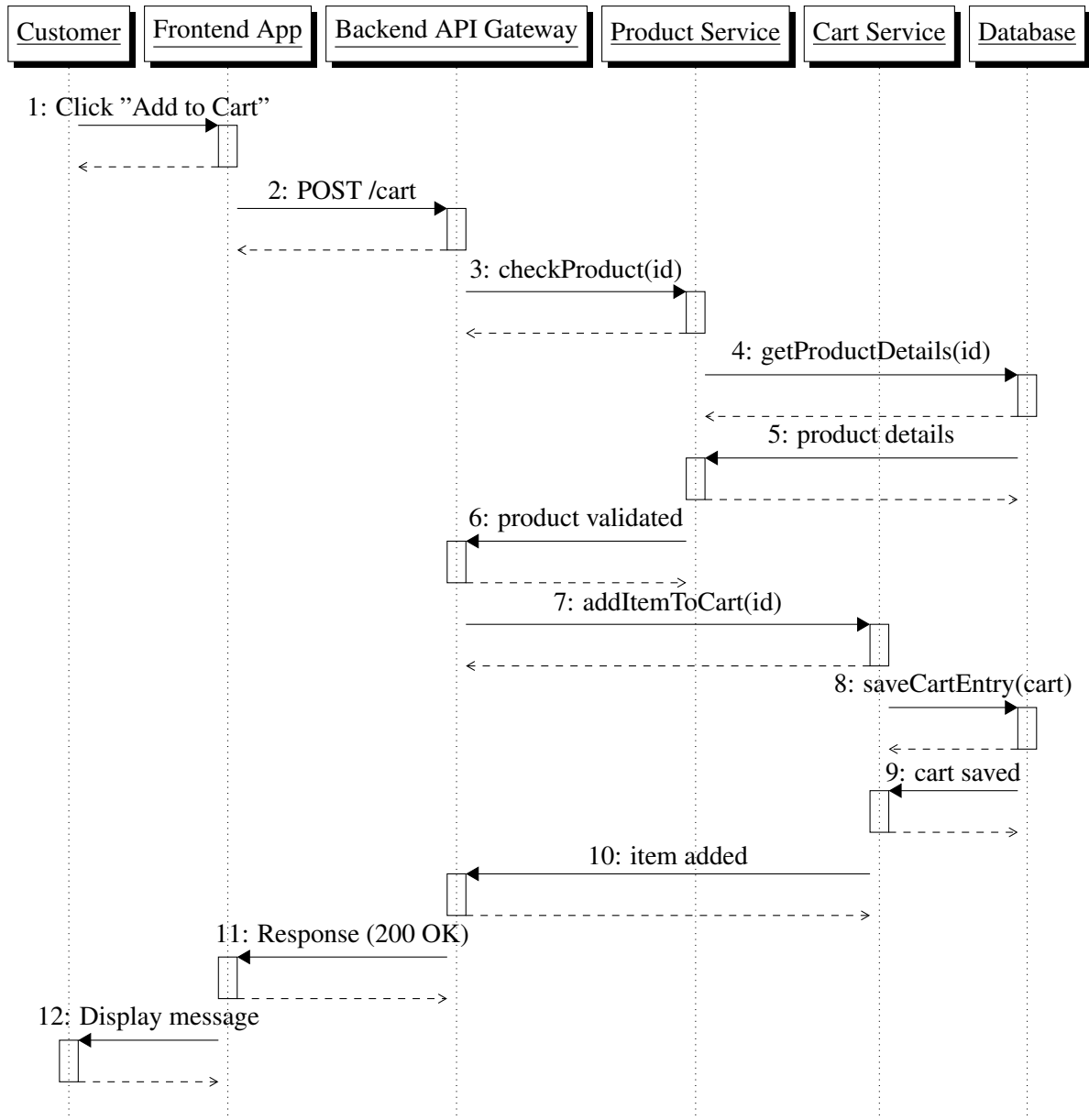


Figure 3: Process View - Sequence Diagram (Add to Cart)

The process view describes the runtime behavior of the system, showing the runtime interaction between system components. This sequence diagram shows how a product is added to the cart. That is, when a customer adds a product to the cart, the front-end sends a request to the back-end API. The back-end checks the product and updates the cart by communicating with the database. This diagram represents the Add to Cart process. This diagram rep-

resents the Add to Cart process. The customer selects a product, the frontend sends a POST /cart request to the backend, the backend validates the product through the Product Service, and the Cart Service saves the cart information in the database.

4 The CODE Part

4.1 Partial UI Code

index.html - Login screen

```
1 <!-- index.html file -->
2 <!DOCTYPE html>
3 <html lang="en">
4 <head>
5     <meta charset="UTF-8">
6     <title>E-Commerce Login - Phase 1</title>
7     <!-- CSS Dosyasini Bagla -->
8     <link rel="stylesheet" href="style.css">
9 </head>
10 <body>
11     <div class="login-container">
12         <h2>Login</h2>
13         <form id="login-form">
14             <label for="email">Email:</label>
15             <input type="email" id="email" placeholder="Enter your email"
16                 required>
17             <label for="password">Password:</label>
18             <input type="password" id="password" placeholder="Enter your
19                 password" required>
20             <button type="submit">Login</button>
21             <div class="register-link">
22                 Don't have an account yet? <a href="register.html">sign
23                 up</a>
24             </div>
25         </form>
26         <p id="message"></p>
27     </div>
28     <!-- JS Dosyasini Bagla -->
29     <script src="script.js"></script>
30 </body>
31 </html>
```

register.html - registration screen

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale
6         =1.0">
7     <title>E-Commerce Register - Phase 1</title>
8     <!-- Stil index.html ile ayn kalacak ekilde uyarland -->
9     <style>
10         body { font-family: Arial, sans-serif; display: flex; justify-
11             content: center; align-items: center; height: 100vh; margin:
12             0; background-color: #f4f4f4; }
```

```

10     .register-container { background: white; padding: 25px; border-
      radius: 8px; box-shadow: 0 2px 10px rgba(0,0,0,0.1); width:
      320px; }
11     h2 { text-align: center; color: #333; }
12     form { display: flex; flex-direction: column; gap: 12px; }
13     label { font-weight: bold; font-size: 14px; }
14     input { padding: 10px; border: 1px solid #ccc; border-radius: 5px
      ; }
15     button { padding: 10px; background-color: #007bff; color: white;
      border: none; border-radius: 5px; cursor: pointer; font-weight
      : bold; }
16     .login-link { text-align: center; margin-top: 15px; font-size: 13
      px; color: #666; }
17     .login-link a { color: #28a745; text-decoration: none; font-
      weight: bold; }
18     </style>
19 </head>
20 <body>
21 <div class="register-container">
22     <h2>Sign Up</h2>
23     <form id="register-form">
24         <label>Full Name:</label>
25         <input type="text" id="name" placeholder="Enter your name"
          required>
26         <label>Email:</label>
27         <input type="email" id="reg-email" placeholder="Enter your email"
          required>
28         <label>Password:</label>
29         <input type="password" id="reg-password" placeholder="Create a
          password" required>
30         <button type="submit">Register</button>
31         <div class="login-link">
32             Already have an account? <a href="index.html">Login</a>
33         </div>
34     </form>
35 </div>
36 <script src="script.js"></script>
37 </body>
38 </html>

```

Products html

```

1     <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>E-Commerce Dashboard</title>
6     <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9     <div class="products-container">
10
11         <div class="navbar">
12             <button onclick="showSection('products')">Products</button>
13             <button onclick="showSection('cart')"> Cart</button>
14             <button onclick="showSection('profile')"> Profile</button>
15             <button onclick="goBack()">Logout</button>
16         </div>
17
18         <div id="products-section">
19             <h2>Product List</h2>
20             <div id="product-list"></div>
21         </div>
22
23         <div id="cart-section" style="display: none;">
24             <h2>Cart</h2>
25             <div id="cart-list"></div>

```

```

26         </div>
27
28         <div id="profile-section" style="display: none;">
29             <h2>Profile</h2>
30             <div id="profile-info"></div>
31         </div>
32
33     </div>
34
35     <script src="products.js"></script>
36 </body>
37 </html>

```

admin.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Admin Panel</title>
6      <link rel="stylesheet" href="style.css">
7  </head>
8  <body>
9      <div class="products-container">
10         <h2>Admin Product Management</h2>
11
12         <form id="add-product-form">
13             <input type="text" id="product-name" placeholder="Product
14                 name" required>
15             <input type="number" id="product-price" placeholder="Product
16                 price" required>
17             <input type="number" id="product-stock" placeholder="Product
18                 stock" required>
19             <button type="submit">Add Product</button>
20         </form>
21
22         <h3>Products</h3>
23         <div id="admin-product-list"></div>
24
25         <button onclick="logout()">Logout</button>
26     </div>
27
28     <script src="admin.js"></script>
29 </body>
30 </html>

```

admin.js

```

1  const API_URL = "http://localhost:3000/api";
2  const userRole = localStorage.getItem("userRole");
3
4  if (userRole !== "admin") {
5      alert("Access denied. Admin only.");
6      window.location.href = "index.html";
7  }
8
9  function loadAdminProducts() {
10      fetch(`${API_URL}/products`)
11          .then(res => res.json())
12          .then(products => {
13              const list = document.getElementById("admin-product-list");
14              list.innerHTML = "";
15
16              products.forEach(product => {
17                  list.innerHTML += `
18                      <div class="product-card">
19                          <h3>${product.name}</h3>

```

```

20         <p>Price: ${product.price}</p>
21         <p>Stock: ${product.stock}</p>
22         <button onclick="deleteProduct(${product.id})">
            Delete</button>
23     </div>
24     `;
25     });
26     });
27 }
28
29 document.getElementById("add-product-form").addEventListener("submit",
    function(e) {
30     e.preventDefault();
31
32     const name = document.getElementById("product-name").value;
33     const price = Number(document.getElementById("product-price").value);
34     const stock = Number(document.getElementById("product-stock").value);
35
36     fetch(`${API_URL}/products`, {
37         method: "POST",
38         headers: { "Content-Type": "application/json" },
39         body: JSON.stringify({ name, price, stock })
40     })
41     .then(res => res.json())
42     .then(data => {
43         alert(data.message);
44         loadAdminProducts();
45         document.getElementById("add-product-form").reset();
46     });
47 });
48
49 function deleteProduct(productId) {
50     fetch(`${API_URL}/products/${productId}`, {
51         method: "DELETE"
52     })
53     .then(data => {
54         alert(data.message);
55         loadAdminProducts();
56         document.getElementById("add-product-form").reset();
57
58         window.scrollTo({
59             top: 0,
60             behavior: "smooth"
61         });
62     });
63 }
64
65 function logout() {
66     localStorage.clear();
67     window.location.href = "index.html";
68 }
69
70 loadAdminProducts();

```

script file-The logic behind form:

```

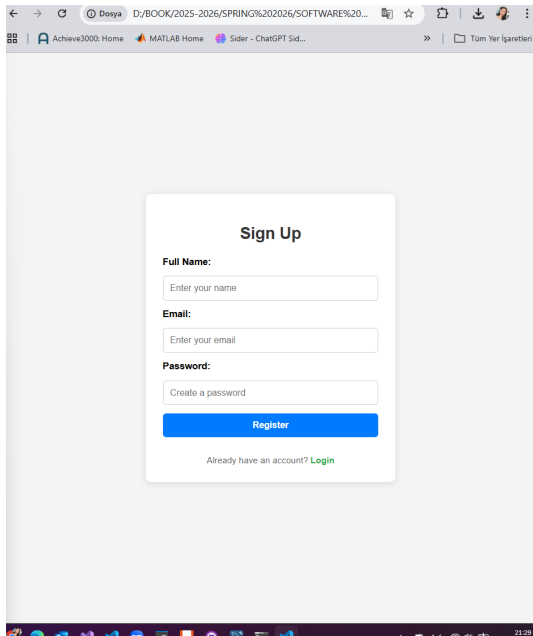
1     const API_URL = "http://localhost:3000/api";
2
3 // LOGIN
4 const loginForm = document.getElementById("login-form");
5
6 if (loginForm) {
7     loginForm.addEventListener("submit", function(e) {
8         e.preventDefault();
9
10         const email = document.getElementById("email").value;
11         const password = document.getElementById("password").value;

```

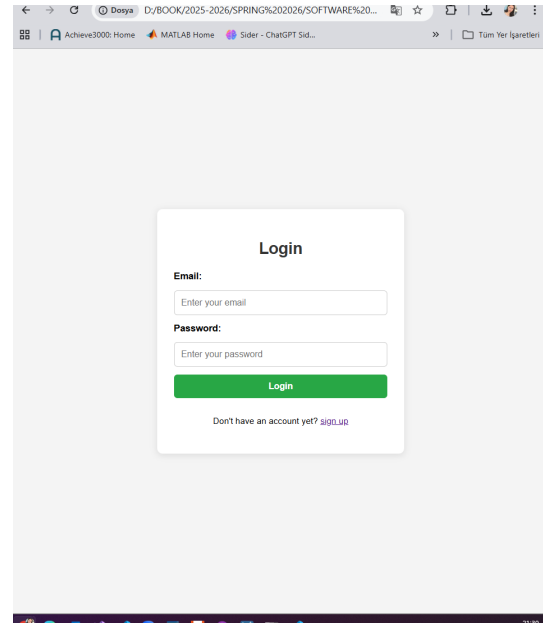
```

12
13     fetch(`${API_URL}/login`, {
14         method: "POST",
15         headers: { "Content-Type": "application/json" },
16         body: JSON.stringify({ email, password })
17     })
18     .then(res => res.json())
19     .then(data => {
20         document.getElementById("message").innerText = data.message;
21
22         if (data.message === "Login successful") {
23             localStorage.setItem("userId", data.user.id);
24             localStorage.setItem("userName", data.user.name);
25             localStorage.setItem("userEmail", data.user.email);
26             localStorage.setItem("userRole", data.user.role);
27
28             if (data.user.role === "admin") {
29                 window.location.href = "admin.html";
30             } else {
31                 window.location.href = "products.html";
32             }
33         }
34     })
35     .catch(() => {
36         document.getElementById("message").innerText =
37             "Connection error (backend is not running)";
38     });
39 });
40 }
41
42 // REGISTER
43 const registerForm = document.getElementById("register-form");
44
45 if (registerForm) {
46     registerForm.addEventListener("submit", function(e) {
47         e.preventDefault();
48
49         const name = document.getElementById("name").value;
50         const email = document.getElementById("reg-email").value;
51         const password = document.getElementById("reg-password").value;
52
53         fetch(`${API_URL}/register`, {
54             method: "POST",
55             headers: { "Content-Type": "application/json" },
56             body: JSON.stringify({ name, email, password })
57         })
58         .then(res => res.json())
59         .then(data => {
60             alert(data.message);
61
62             if (data.message === "Registration successful") {
63                 window.location.href = "index.html";
64             }
65         })
66         .catch(() => {
67             alert("Connection error (backend is not running)");
68         });
69     });
70 }

```



(a) Sign Up Screen



(b) Login Screen

Figure 4: User Interface Screens

Products.js

```

1   const API_URL = "http://localhost:3000/api";
2
3   const userId = localStorage.getItem("userId");
4   const userName = localStorage.getItem("userName");
5   const userEmail = localStorage.getItem("userEmail");
6
7   if (!userId) {
8     alert("Please login first");
9     window.location.href = "index.html";
10  }
11
12  function showSection(section) {
13    document.getElementById("products-section").style.display = "none";
14    document.getElementById("cart-section").style.display = "none";
15    document.getElementById("profile-section").style.display = "none";
16
17    if (section === "products") {
18      document.getElementById("products-section").style.display = "
19        block";
20      loadProducts();
21    }
22
23    if (section === "cart") {
24      document.getElementById("cart-section").style.display = "block";
25      loadCart();
26    }
27
28    if (section === "profile") {
29      document.getElementById("profile-section").style.display = "block
30        ";
31      loadProfile();
32    }
33  }
34
35  function loadProfile() {
36    fetch(`${API_URL}/cart/${userId}`)
37      .then(res => res.json())

```

```

36     .then(cart => {
37         let totalItems = 0;
38         let totalPrice = 0;
39
40         cart.forEach(item => {
41             totalItems += item.quantity;
42             totalPrice += item.price * item.quantity;
43         });
44
45         document.getElementById("profile-info").innerHTML = `
46             <div class="profile-card">
47                 
49                 <h3> ${userName}</h3>
50                 <p><strong>Email:</strong> ${userEmail}</p>
51                 <p><strong>User ID:</strong> ${userId}</p>
52
53                 <hr>
54
55                 <p>          Total Items in Cart: <strong>${totalItems
56                     }</strong></p>
57                 <p>          Total Cart Value: <strong>${totalPrice}</
58                     strong></p>
59             </div>
60         `;
61     });
62 }
63
64 function loadProducts() {
65     fetch(`${API_URL}/products`)
66     .then(res => res.json())
67     .then(products => {
68         const productList = document.getElementById("product-list");
69         productList.innerHTML = "";
70
71         products.forEach(product => {
72             productList.innerHTML += `
73                 <div class="product-card">
74                     <h3>${product.name}</h3>
75                     <p>Price: ${product.price}</p>
76                     <p>Stock: ${product.stock}</p>
77                     <button onclick="addToCart(${product.id})"
78                         ${product.stock <= 0 ? "disabled" : ""}>
79                         ${product.stock <= 0 ? "Out of Stock" : "Add
80                             to Cart"}
81                     </button>
82                 </div>
83             `;
84         });
85     });
86 }
87
88 function addToCart(productId) {
89     fetch(`${API_URL}/cart`, {
90         method: "POST",
91         headers: {
92             "Content-Type": "application/json"
93         },
94         body: JSON.stringify({
95             productId: productId,
96             userId: userId
97         })
98     })
99     .then(res => res.json())
100     .then(data => {
101         alert(data.message);
102         loadProducts();
103     });
104 }

```

```

99         loadCart();
100     });
101 }
102
103 function loadCart() {
104     fetch(`${API_URL}/cart/${userId}`)
105         .then(res => res.json())
106         .then(cart => {
107             const cartList = document.getElementById("cart-list");
108             cartList.innerHTML = "";
109
110             if (cart.length === 0) {
111                 cartList.innerHTML = "<p>Your cart is empty.</p>";
112                 return;
113             }
114
115             let total = 0;
116
117             cart.forEach(item => {
118                 total += item.price * item.quantity;
119
120                 cartList.innerHTML += `
121                     <div class="cart-item">
122                         <p>${item.name} - ${item.price} - Quantity: ${
123                             item.quantity}</p>
124                         <button onclick="removeFromCart(${item.id})">
125                             Remove</button>
126                     </div>
127                 `;
128             });
129
130             cartList.innerHTML += `<h3>Total: ${total}</h3>`;
131         });
132 }
133
134 function removeFromCart(productId) {
135     fetch(`${API_URL}/cart`, {
136         method: "DELETE",
137         headers: {
138             "Content-Type": "application/json"
139         },
140         body: JSON.stringify({
141             productId: productId,
142             userId: userId
143         })
144     })
145     .then(res => res.json())
146     .then(data => {
147         alert(data.message);
148         loadProducts();
149         loadCart();
150     });
151 }
152
153 function goBack() {
154     localStorage.removeItem("userId");
155     localStorage.removeItem("userName");
156     localStorage.removeItem("userEmail");
157     localStorage.removeItem("userRole");
158     window.location.href = "index.html";
159 }
160
161 showSection("products");

```

css file- The design of both pages is shared:

```

1 body {

```

```

2     font-family: Arial, sans-serif;
3     display: flex;
4     justify-content: center;
5     align-items: center;
6     height: 100vh;
7     margin: 0;
8     background-color: #f4f4f4;
9 }
10
11 .login-container {
12     background: white;
13     padding: 25px;
14     border-radius: 8px;
15     box-shadow: 0 2px 10px rgba(0,0,0,0.1);
16     width: 320px;
17 }
18
19 h2 {
20     text-align: center;
21     color: #333;
22     margin-bottom: 20px;
23 }
24
25 form {
26     display: flex;
27     flex-direction: column;
28     gap: 12px;
29 }
30
31 label {
32     font-weight: bold;
33     font-size: 14px;
34 }
35
36 input {
37     padding: 10px;
38     border: 1px solid #ccc;
39     border-radius: 5px;
40     outline: none;
41 }
42
43 button {
44     padding: 10px;
45     background-color: #28a745;
46     color: white;
47     border: none;
48     border-radius: 5px;
49     cursor: pointer;
50     font-weight: bold;
51 }
52
53 .register-link {
54     text-align: center;
55     margin-top: 15px;
56     font-size: 13px;
57 }
58
59 #message {
60     text-align: center;
61     margin-top: 15px;
62     color: red;
63 }
64
65 .products-container {
66     background: white;
67     padding: 25px;
68     border-radius: 8px;

```

```

69     box-shadow: 0 2px 10px rgba(0,0,0,0.1);
70     width: 500px;
71 }
72
73 .product-card {
74     border: 1px solid #ccc;
75     padding: 12px;
76     margin: 10px 0;
77     border-radius: 6px;
78 }
79
80 .cart-item {
81     background-color: #f4f4f4;
82     padding: 8px;
83     margin: 8px 0;
84     border-radius: 5px;
85 }
86
87 .navbar {
88     display: flex;
89     gap: 10px;
90     margin-bottom: 20px;
91     justify-content: center;
92 }
93
94 .navbar button {
95     background-color: #007bff;
96 }
97
98 #profile-info {
99     background-color: #f4f4f4;
100     padding: 15px;
101     border-radius: 6px;
102 }
103
104 .profile-card {
105     background: #f9f9f9;
106     padding: 20px;
107     border-radius: 8px;
108     text-align: center;
109 }
110
111 .profile-card h3 {
112     margin-bottom: 10px;
113 }
114
115 .profile-card hr {
116     margin: 15px 0;
117 }
118
119 #add-product-form {
120     position: sticky;
121     top: 10px;
122     background: white;
123     z-index: 10;
124     padding-bottom: 10px;
125 }
126
127 .products-container {
128     margin-top: 30px;
129     max-height: 85vh;
130     overflow-y: auto;
131 }
132
133 .navbar {
134     position: sticky;
135     top: 0;

```

```

136     background: white;
137     z-index: 100;
138     padding: 15px 0;
139     border-bottom: 1px solid #ddd;
140 }
141
142 #products-section,
143 #cart-section,
144 #profile-section {
145     padding-top: 20px;
146 }
147 #product-list,
148 #admin-product-list {
149     display: grid;
150     grid-template-columns: repeat(2, 1fr);
151     gap: 15px;
152 }
153
154 .product-card {
155     margin: 0;
156 }
157 @media (max-width: 700px) {
158     #product-list,
159     #admin-product-list {
160         grid-template-columns: 1fr;
161     }
162 }

```

4.2 Backend Code

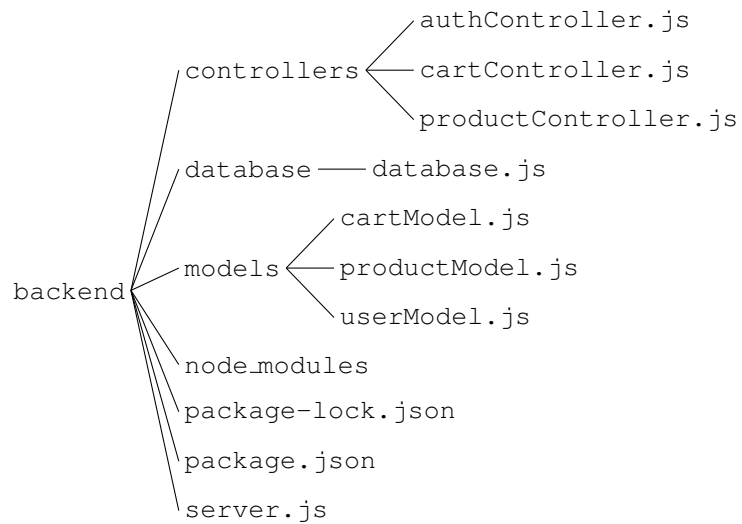


Figure 5: Backend Folder Structure

4.2.1 controllers

authController.js

```

1     const UserModel = require("../models/userModel");
2
3     function register(req, res) {
4         const { name, email, password } = req.body;
5
6         const existingUser = UserModel.findUserByEmail(email);
7

```

```

8     if (existingUser) {
9         return res.status(400).json({
10             message: "User already exists"
11         });
12     }
13
14     const user = UserModel.createUser(name, email, password);
15
16     res.status(201).json({
17         message: "Registration successful",
18         user: user
19     });
20 }
21
22 function login(req, res) {
23     const { email, password } = req.body;
24
25     const user = UserModel.findUserByEmail(email);
26
27     if (!user || user.password !== password) {
28         return res.status(401).json({
29             message: "Invalid email or password"
30         });
31     }
32
33     res.status(200).json({
34         message: "Login successful",
35         user: user
36     });
37 }
38
39 module.exports = {
40     register,
41     login
42 };

```

cartController.js

```

1     const ProductModel = require("../models/productModel");
2     const CartModel = require("../models/cartModel");
3
4     function addToCart(req, res) {
5         const { productId, userId } = req.body;
6
7         const product = ProductModel.findProductById(productId);
8
9         if (!product) {
10             return res.status(404).json({
11                 message: "Product not found"
12             });
13         }
14
15         if (product.stock <= 0) {
16             return res.status(400).json({
17                 message: "Product is out of stock"
18             });
19         }
20
21         const cartItem = CartModel.addToCart(userId, product);
22         ProductModel.decreaseStock(productId);
23
24         res.status(200).json({
25             message: "Product added to cart",
26             cartItem: cartItem
27         });
28     }
29

```

```

30 function getCart(req, res) {
31     const { userId } = req.params;
32     const cart = CartModel.getCart(userId);
33
34     res.status(200).json(cart);
35 }
36
37 function removeFromCart(req, res) {
38     const { productId, userId } = req.body;
39
40     const removedItem = CartModel.removeFromCart(userId, productId);
41
42     if (!removedItem) {
43         return res.status(404).json({
44             message: "Product not found in cart"
45         });
46     }
47
48     res.status(200).json({
49         message: "Product removed from cart",
50         removedItem: removedItem
51     });
52 }
53
54 module.exports = {
55     addToCart,
56     getCart,
57     removeFromCart
58 };

```

productController.js

```

1     const ProductModel = require("../models/productModel");
2
3 function getProducts(req, res) {
4     const products = ProductModel.getAllProducts();
5     res.status(200).json(products);
6 }
7
8 function addProduct(req, res) {
9     const { name, price, stock } = req.body;
10
11     if (!name || !price || stock === undefined) {
12         return res.status(400).json({
13             message: "Product name, price and stock are required"
14         });
15     }
16
17     const product = ProductModel.addProduct(name, price, stock);
18
19     res.status(201).json({
20         message: "Product added successfully",
21         product: product
22     });
23 }
24
25 function deleteProduct(req, res) {
26     const { id } = req.params;
27
28     const deletedProduct = ProductModel.deleteProduct(id);
29
30     if (!deletedProduct) {
31         return res.status(404).json({
32             message: "Product not found"
33         });
34     }
35 }

```

```

36     res.status(200).json({
37         message: "Product deleted successfully",
38         product: deletedProduct
39     });
40 }
41
42 module.exports = {
43     getProducts,
44     addProduct,
45     deleteProduct
46 };

```

4.2.2 models

cartModel

```

1  const carts = {};
2
3  function addToCart(userId, product) {
4      if (!carts[userId]) {
5          carts[userId] = [];
6      }
7
8      const userCart = carts[userId];
9
10     const existingItem = userCart.find(item => item.id === product.id);
11
12     if (existingItem) {
13         existingItem.quantity += 1;
14         return existingItem;
15     }
16
17     const cartItem = {
18         id: product.id,
19         name: product.name,
20         price: product.price,
21         quantity: 1
22     };
23
24     userCart.push(cartItem);
25     return cartItem;
26 }
27
28 function getCart(userId) {
29     return carts[userId] || [];
30 }
31
32 function removeFromCart(userId, productId) {
33     const userCart = carts[userId];
34
35     if (!userCart) {
36         return null;
37     }
38
39     const item = userCart.find(item => item.id === Number(productId));
40
41     if (!item) {
42         return null;
43     }
44
45     if (item.quantity > 1) {
46         item.quantity -= 1;
47         return item;
48     }
49 }

```

```

50     const index = userCart.findIndex(item => item.id === Number(productId
51         ));
52     const removedItem = userCart.splice(index, 1);
53     return removedItem[0];
54 }
55
56 module.exports = {
57     addToCart,
58     getCart,
59     removeFromCart
60 };

```

productModel

```

1  const products = [
2      { id: 1, name: "Laptop", price: 1500, stock: 5 },
3      { id: 2, name: "Phone", price: 900, stock: 3 },
4      { id: 3, name: "Headphones", price: 100, stock: 10 }
5  ];
6
7  function getAllProducts() {
8      return products;
9  }
10
11 function findProductById(id) {
12     return products.find(product => product.id === Number(id));
13 }
14
15 function addProduct(name, price, stock) {
16     const product = {
17         id: products.length + 1,
18         name: name,
19         price: Number(price),
20         stock: Number(stock)
21     };
22
23     products.push(product);
24     return product;
25 }
26
27 function deleteProduct(id) {
28     const index = products.findIndex(product => product.id === Number(id)
29         );
30
31     if (index === -1) {
32         return null;
33     }
34
35     const deletedProduct = products.splice(index, 1);
36     return deletedProduct[0];
37 }
38
39 function decreaseStock(productId) {
40     const product = findProductById(productId);
41
42     if (!product || product.stock <= 0) {
43         return false;
44     }
45
46     product.stock -= 1;
47     return true;
48 }
49
50 module.exports = {
51     getAllProducts,
52     findProductById,

```

```

52     addProduct,
53     deleteProduct,
54     decreaseStock
55 };

```

userModel

```

1  const users = [
2    {
3      id: 1,
4      name: "Admin",
5      email: "admin@test.com",
6      password: "admin123",
7      role: "admin"
8    }
9  ];
10
11 function createUser(name, email, password) {
12   const user = {
13     id: users.length + 1,
14     name: name,
15     email: email,
16     password: password,
17     role: "customer"
18   };
19
20   users.push(user);
21   return user;
22 }
23
24 function findUserByEmail(email) {
25   return users.find(user => user.email === email);
26 }
27
28 module.exports = {
29   createUser,
30   findUserByEmail
31 };

```

4.2.3 server.js

```

1  const express = require("express");
2  const cors = require("cors");
3
4  const authController = require("./controllers/authController");
5  const productController = require("./controllers/productController");
6  const cartController = require("./controllers/cartController");
7
8  const app = express();
9
10 app.use(cors());
11 app.use(express.json());
12
13 app.post("/api/register", authController.register);
14 app.post("/api/login", authController.login);
15
16 app.get("/api/products", productController.getProducts);
17 app.post("/api/products", productController.addProduct);
18 app.delete("/api/products/:id", productController.deleteProduct);
19
20 app.post("/api/cart", cartController.addToCart);
21 app.get("/api/cart/:userId", cartController.getCart);
22 app.delete("/api/cart", cartController.removeFromCart);
23

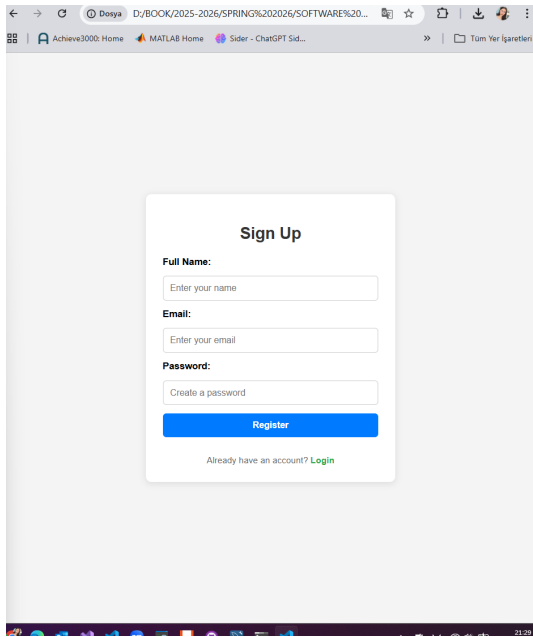
```

```

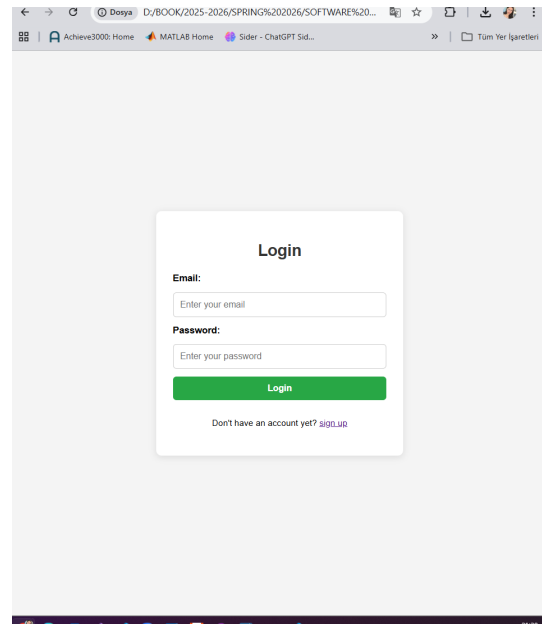
24 app.get("/api/health", (req, res) => {
25     res.json({ message: "Backend is running" });
26 });
27
28 app.listen(3000, () => {
29     console.log("Backend server is running on port 3000");
30 });

```

5 Web Application



(a) Sign Up Screen



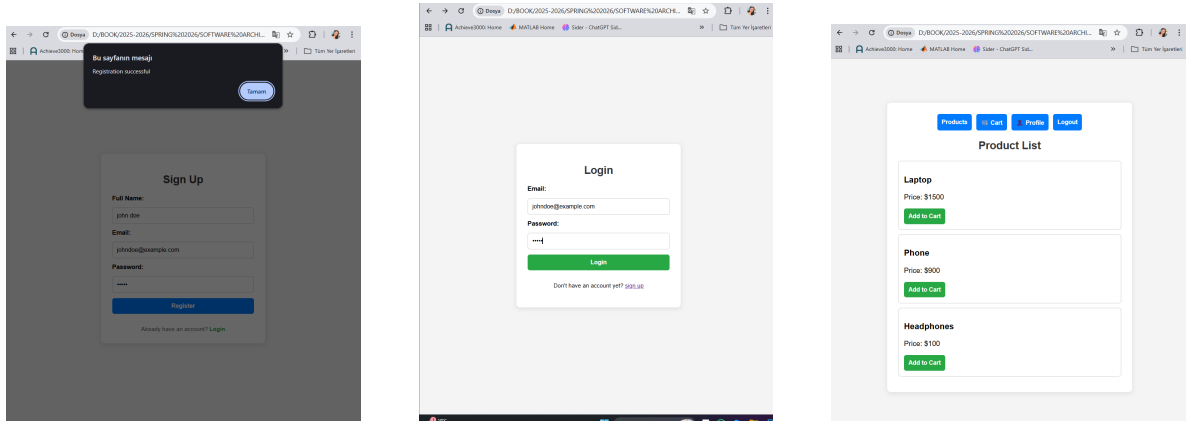
(b) Login Screen

Figure 6: User Interface Screens

5.1 User/Customer login

Users can create an account and log in from this screen. After logging in, they will see a screen like this:

John Doe creates and logs into a customer account:



(a) Signup

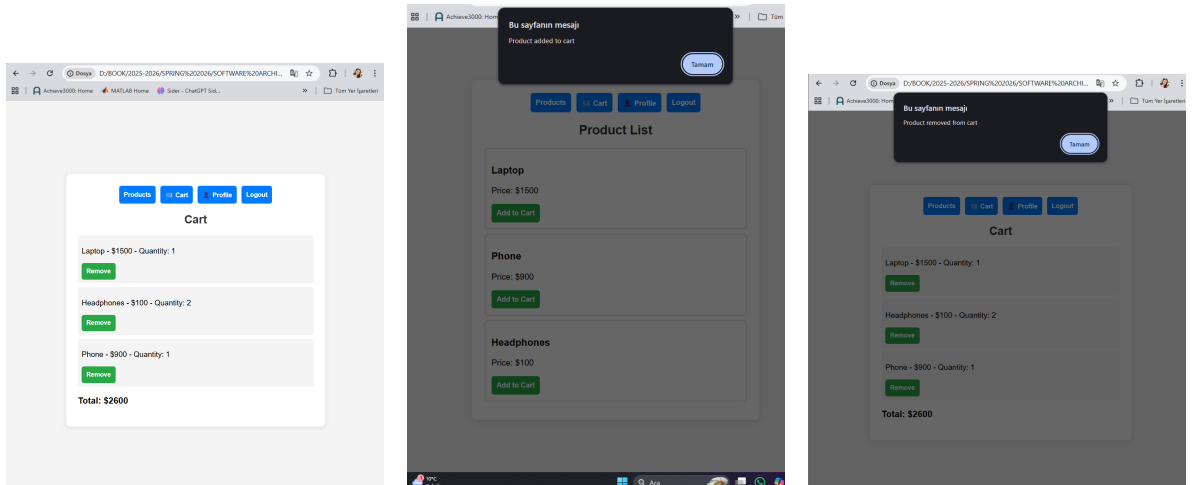
(b) Login

(c) Home

Figure 7: User Interface Screens

The user can then switch between the following screens: products, cart, profile, logout.

Here, the user viewed their cart and added and removed products:



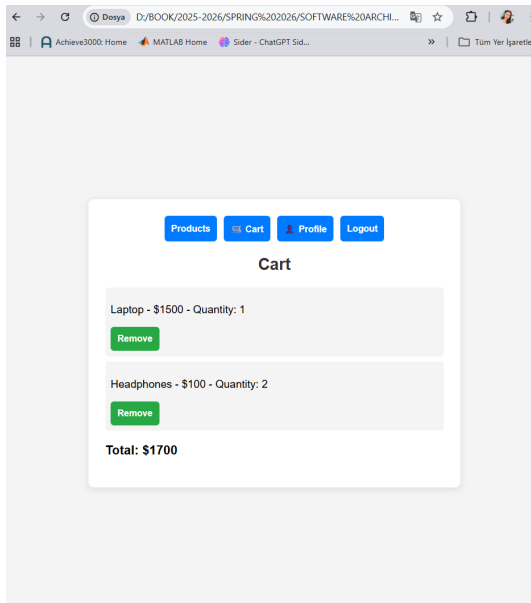
(a) Shopping Cart Viewing

(b) Adding a Product

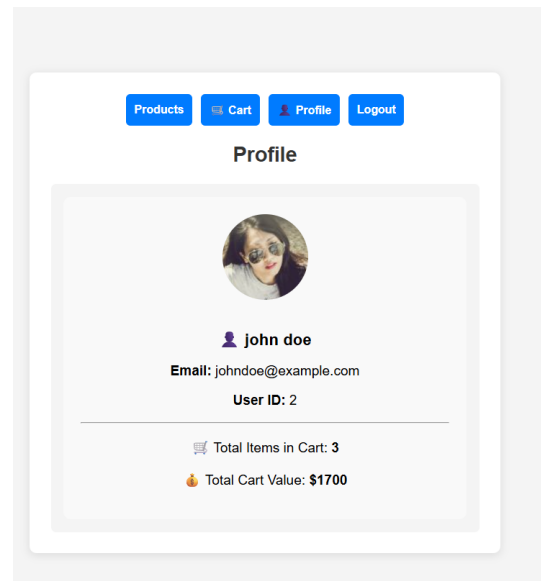
(c) Product Removal

Figure 8: Cart Operations

User's current shopping cart and profile after removing products:



(a) Product removed from cart



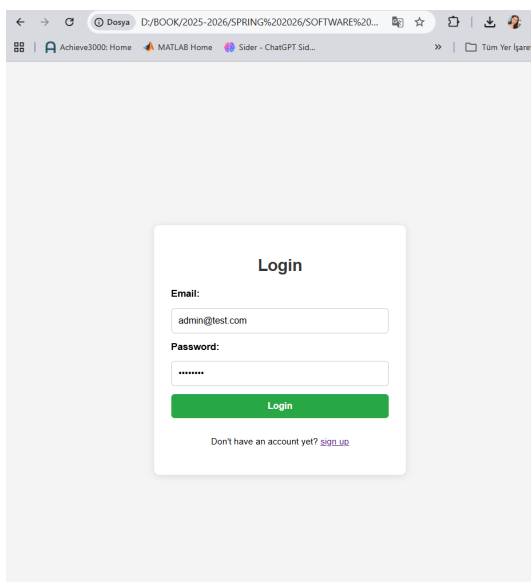
(b) Profil

Figure 9: User Screens

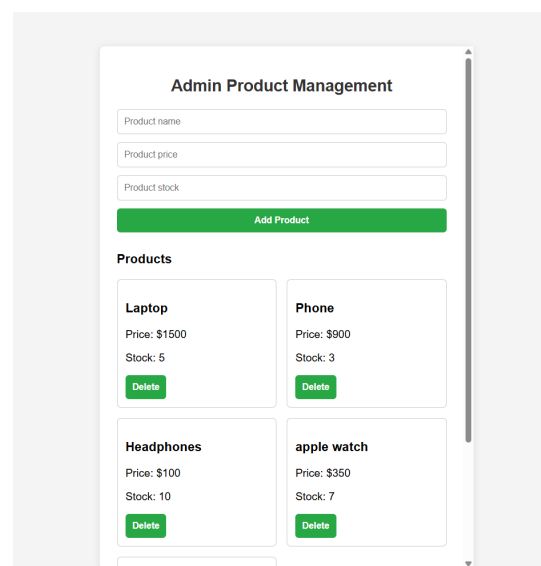
5.1.1 Admin login

Administrators can log in using the email address "admin@test.com" and the password "admin123" to add and remove products.

Here is the admin login screen:



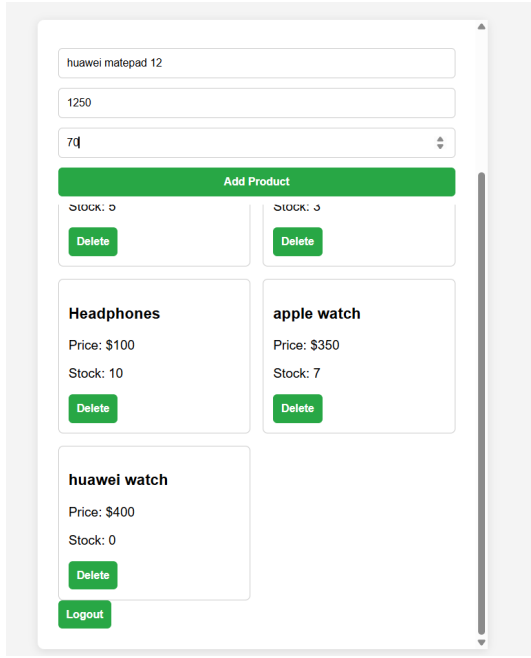
(a) Admin Login Screen



(b) Admin Home Screen

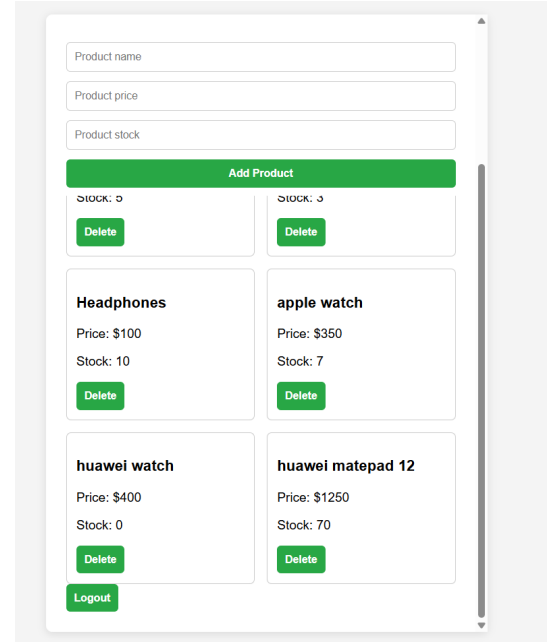
Figure 10: Admin Interface Screens

Admin can add products from here:



The screenshot shows the 'Add Product' form in the admin interface. It includes input fields for 'Product name' (filled with 'huawei matepad 12'), 'Product price' (filled with '1250'), and 'Product stock' (filled with '70'). Below these fields is a green 'Add Product' button. Underneath, there are two columns of product cards. The first column contains 'Headphones' (Price: \$100, Stock: 10) and 'huawei watch' (Price: \$400, Stock: 0). The second column contains 'apple watch' (Price: \$350, Stock: 7). Each card has a green 'Delete' button. At the bottom left, there is a green 'Logout' button.

(a) Ürün Ekleme

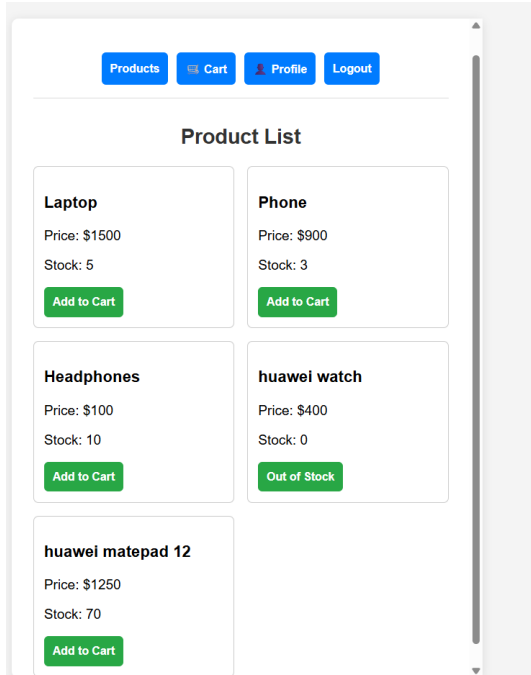


The screenshot shows the 'Product List' in the admin interface. It displays a list of products with their details and a green 'Delete' button for each. The products are: 'Headphones' (Price: \$100, Stock: 10), 'apple watch' (Price: \$350, Stock: 7), 'huawei watch' (Price: \$400, Stock: 0), and 'huawei matepad 12' (Price: \$1250, Stock: 70). A green 'Logout' button is at the bottom left.

(b) Ürün Eklendi

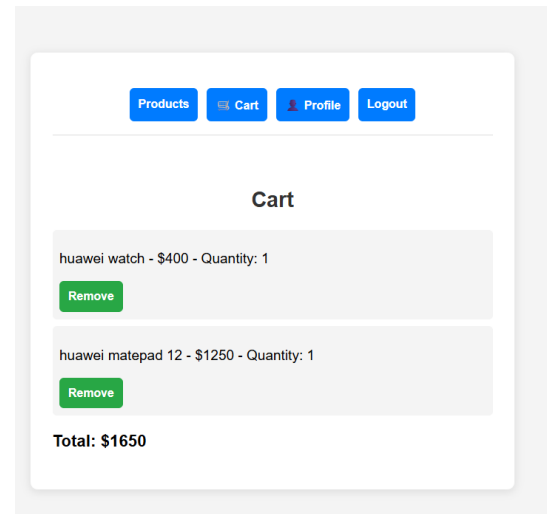
Figure 11: Admin Product Management

And finally, the product added by the admin can be viewed by the user, added to the cart, or removed from the cart:



The screenshot shows the 'Product List' in the user interface. It displays a list of products with their details and a green 'Add to Cart' button for each. The products are: 'Laptop' (Price: \$1500, Stock: 5), 'Phone' (Price: \$900, Stock: 3), 'Headphones' (Price: \$100, Stock: 10), and 'huawei watch' (Price: \$400, Stock: 0). A green 'Add to Cart' button is at the bottom left.

(a) Yeni Ürün Görüntüleme



The screenshot shows the 'Cart' in the user interface. It displays a list of items in the cart with their details and a green 'Remove' button for each. The items are: 'huawei watch - \$400 - Quantity: 1' and 'huawei matepad 12 - \$1250 - Quantity: 1'. A green 'Remove' button is at the bottom left. The total price is displayed as 'Total: \$1650'.

(b) Sepete Ekleme

Figure 12: User Product Interaction